

(5) 读栈顶元素

```
bool GetTop(SqStack S, ElemType &x) {
    if (S.top == -1)           // 栈空, 报错
        return false;
    x = S.data[S.top];         // x 记录栈顶元素
    return true;
}
```

该操作不改变栈的状态, 栈顶元素依然保留在栈中。

注 意

此处, top 指向栈顶元素本身。因此入栈为 $S.data[++S.top]=x$, 出栈为 $x=S.data[S.top--]$ 。若 top 初始化为 0 (指向栈顶元素的下一个位置), 则入栈变为 $S.data[S.top++]=x$, 出栈变为 $x=S.data[--S.top]$, 相应的判空、判满条件也随之改变。请读者仔细体会差异, 做题时务必根据题目设定灵活应对。

3. 共享栈

利用栈底位置相对固定的特性, 可让两个顺序栈共享同一段一维数组空间, 将两个栈的栈底分别置于数组两端, 栈顶向中间延伸, 如图 3.3 所示。

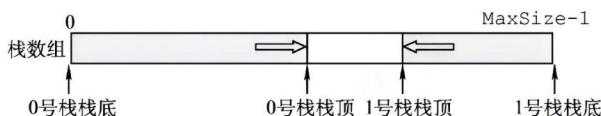


图 3.3 两个顺序栈共享存储空间

0号栈栈顶指针为 top_0 , 1号栈栈顶指针为 top_1 , 均指向各自的栈顶元素; 初始时 $top_0 = -1$ (0号栈空), $top_1 = \text{MaxSize}$ (1号栈空); 栈满条件为 $top_1 - top_0 == 1$ (两栈顶相邻)。当0号栈入栈时, top_0 先加1, 再赋值; 当1号栈入栈时, top_1 先减1, 再赋值; 出栈操作的顺序相反。

共享栈能更高效地利用存储空间, 两个栈的空间可动态调节, 仅当整个数组被占满时, 才发生栈溢出。其存取数据的时间复杂度均为 $O(1)$, 对存取效率无影响。

3.1.3 栈的链式存储结构

采用链式存储的栈称为链栈。其优点是便于动态分配存储空间, 不存在栈满溢出的问题, 且在多栈共存的场景下能更灵活地利用内存。通常使用单链表实现链栈, 并且规定所有操作均在链表的表头进行。此处约定链栈不带头结点, 栈顶指针 $Lhead$ 直接指向栈顶元素, 如图 3.4 所示。

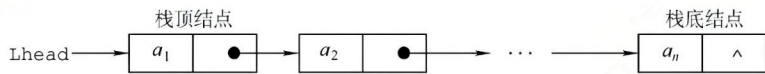


图 3.4 栈的链式存储

链栈的类型可定义为

```
typedef struct LinkNode {
    ElemType data;           // 数据域
    struct LinkNode *next;   // 指针域
} LiStack;                  // 链栈类型定义
```

由于采用链式存储, 结点的插入与删除操作非常高效。链栈的操作与单链表类似, 入栈和出栈均在表头进行。注意, 对于带头结点的链栈, 其初始化、判空及操作细节会有所不同; 而本节所述实现基于无头结点的设计, 读者应根据实际需求灵活调整实现方式。

3.1.4 本节试题精选

一、单项选择题

01. 栈和队列具有相同的 ()。
A. 抽象数据类型 B. 逻辑结构 C. 存储结构 D. 运算
02. 栈是一种 ()。
A. 顺序存储的线性结构 B. 链式存储的非线性结构
C. 限制存取点的线性结构 D. 限制存取点的非线性结构
03. 下列选项中, () 不是栈的基本操作。
A. 删除栈顶元素 B. 删除栈底元素
C. 判断栈是否为空 D. 将栈置为空栈
04. 设用数组 $a[n]$ 存储一个栈, 初始栈顶指针 $top=-1$, 则元素 x 入栈的操作是 ()。
A. $a[--top]=x$ B. $a[top--]=x$ C. $a[++top]=x$ D. $a[top++]=x$
05. 设用数组 $data[1..n]$ 存储一个栈, 初始栈顶指针 $top=1$, 则元素 x 入栈的操作是 ()。
A. $data[top--]=x$ B. $data[top++]=x$
C. $data[--top]=x$ D. $data[++top]=x$
06. 设用数组 $data[1..n]$ 存储一个栈, 初始栈顶指针 $top=n+1$, 则元素 x 入栈的操作是 ()。
A. $data[--top]=x$ B. $data[top++]=x$
C. $data[top--]=x$ D. $data[++top]=x$
07. 设有一个空栈, 栈顶指针为 $1000H$, 栈向高地址方向增长, 每个元素占一个存储单元, 执行 Push、Push、Pop、Push、Pop、Push、Pop、Push 操作后, 栈顶指针为 ()。
A. $1002H$ B. $1003H$ C. $1004H$ D. $1005H$
08. 和顺序栈相比, 链栈有一个比较明显的优势, 即 ()。
A. 通常不会出现栈满的情况 B. 通常不会出现栈空的情况
C. 插入操作更容易实现 D. 删除操作更容易实现
09. 设链表不带头结点且所有操作均在表头进行, 则下列最不适合作为链栈的是 ()。
A. 只有表头结点指针, 没有表尾指针的双向循环链表
B. 只有表尾结点指针, 没有表头指针的双向循环链表
C. 只有表头结点指针, 没有表尾指针的单向循环链表
D. 只有表尾结点指针, 没有表头指针的单向循环链表
10. 向一个栈顶指针为 top 的链栈 (不带头结点) 中插入一个 x 结点, 则执行 ()。
A. $top->next=x$ B. $x->next=top->next; top->next=x$
C. $x->next=top; top=x$ D. $x->next=top; top=top->next$
11. 链栈 (不带头结点) 执行 Pop 操作, 并将出栈的元素存在 x 中, 应该执行 ()。
A. $x=top; top=top->next$
B. $x=top->data$
C. $top=top->next; x=top->data$
D. $x=top->data; top=top->next$
12. 经过以下栈的操作后, 变量 x 的值为 ()。
 $InitStack(st); Push(st, a); Push(st, b); Pop(st, x); GetTop(st, x);$
A. a B. b C. $NULL$ D. $false$
13. 3 个不同元素依次入栈, 能得到 () 种不同的出栈序列。

- A. 4 B. 5 C. 6 D. 7
14. 设 a, b, c, d, e, f 以所给的次序入栈, 若在入栈操作时, 允许出栈操作, 则下面不会出现的出栈序列为 ()。
- A. $fedcba$ B. $bcafed$ C. $dcefb a$ D. $cabdef$
15. 4 个元素依次入栈的次序为 $abcd$, 则以 cd 开头的出栈序列的个数为 ()。
- A. 1 B. 2 C. 3 D. 4
16. 用 S 表示入栈操作, 用 X 表示出栈操作, 若元素的入栈顺序是 1234, 为了得到 1342 的出栈顺序, 相应的 S 和 X 的操作序列为 ()。
- A. $SXSXSXSX$ B. $SSSXSXSX$ C. $SXSXSXSX$ D. $SXSXSXSX$
17. 若栈的输入序列是 $1, 2, 3, \dots, n$, 输出序列的第一个元素是 n , 则第 i 个输出元素是 ()。
- A. 不确定 B. $n-i$ C. $n-i-1$ D. $n-i+1$
18. 若栈的输入序列是 $1, 2, 3, \dots, n$, 输出序列的第一个元素是 i , 则第 j 个输出元素是 ()。
- A. $i-j-1$ B. $i-j$ C. $j-i+1$ D. 不确定
19. 某栈的输入序列为 a, b, c, d , 下面的 4 个序列中, 不可能为其输出序列的是 ()。
- A. a, b, c, d B. c, b, d, a C. d, c, a, b D. a, c, b, d
20. 若栈的输入序列是 $P_1, P_2, \dots, P_n$, 输出序列是 $1, 2, 3, \dots, n$, 若 $P_3=1$, 则 P_1 的值 ()。
- A. 可能是 2 B. 一定是 2 C. 不可能是 2 D. 不可能是 3
21. 若栈的输入序列是 $P_1, P_2, \dots, P_n$, 输出序列是 $1, 2, 3, \dots, n$, 若 $P_3=3$, 则 P_1 的值 ()。
- A. 可能是 2 B. 不可能是 1 C. 一定是 1 D. 一定是 2
22. 已知栈的入栈序列是 $1, 2, 3, 4$, 其出栈序列为 P_1, P_2, P_3, P_4 , 则 P_2, P_4 不可能是 ()。
- A. 2, 4 B. 2, 1 C. 4, 3 D. 3, 4
23. 设栈的初始状态为空, 当字符序列 “nl_” 作为栈的输入时, 输出长度为 3, 且可用作 C 语言标识符的序列有 () 个。
- A. 4 B. 5 C. 3 D. 6
24. 采用共享栈的好处是 ()。
- A. 减少存取时间, 降低发生上溢的可能
B. 节省存储空间, 降低发生上溢的可能
C. 减少存取时间, 降低发生下溢的可能
D. 节省存储空间, 降低发生下溢的可能
25. 设有一个顺序共享栈 $Share[0:n-1]$, 其中第一个栈顶指针 $top1$ 的初值为 -1, 第二个栈顶指针 $top2$ 的初值为 n , 则判断共享栈满的条件是 ()。
- A. $top2-top1==1$ B. $top1-top2==1$
C. $top1==top2$ D. 都不对
26. 【2009 统考真题】设栈 S 和队列 Q 的初始状态均为空, 元素 $abcdefg$ 依次入栈 S 。若每个元素出栈后立即进入队列 Q , 且 7 个元素出队的顺序是 $bdcfeag$, 则栈 S 的容量至少是 ()。
- A. 1 B. 2 C. 3 D. 4
27. 【2010 统考真题】若元素 a, b, c, d, e, f 依次入栈, 允许入栈、出栈操作交替进行, 但不允许连续 3 次进行出栈操作, 不可能得到的出栈序列是 ()。
- A. $dcebf a$ B. $cbdaef$ C. $bcaef d$ D. $afedcb$
28. 【2011 统考真题】元素 a, b, c, d, e 依次进入初始为空的栈中, 若元素入栈后可停留、可出栈, 直到所有元素都出栈, 则在所有可能的出栈序列中, 以元素 d 开头的序列个数是 ()。

- A. 3 B. 4 C. 5 D. 6

29. 【2013 统考真题】一个栈的入栈序列为 $1, 2, 3, \dots, n$, 出栈序列是 $P_1, P_2, P_3, \dots, P_n$ 。若 $P_2=3$, 则 P_3 可能取值的个数是 ()。

- A. $n-3$ B. $n-2$ C. $n-1$ D. 无法确定

30. 【2020 统考真题】对空栈 S 进行 Push 和 Pop 操作, 入栈序列为 a, b, c, d, e , 经过 Push、Push、Pop、Push、Pop、Push、Push、Pop 操作后得到的出栈序列是 ()。

- A. b, a, c B. b, a, e C. b, c, a D. b, c, e

31. 【2022 统考真题】给定有限符号集 S , in 和 out 均为 S 中所有元素的任意排列。对于初始为空的栈 ST , 下列叙述中, 正确的是 ()。

- A. 若 in 是 ST 的入栈序列, 则不能判断 out 是否为其可能的出栈序列
B. 若 out 是 ST 的出栈序列, 则不能判断 in 是否为其可能的入栈序列
C. 若 in 是 ST 的入栈序列, out 是对应 in 的出栈序列, 则 in 与 out 一定不同
D. 若 in 是 ST 的入栈序列, out 是对应 in 的出栈序列, 则 in 与 out 可能互为倒序

二、综合应用题

01. 有 5 个元素, 其入栈次序为 A, B, C, D, E , 在各种可能的出栈次序中, 第一个出栈元素为 C 且第二个出栈元素为 D 的出栈序列有哪几个?

02. 若元素的入栈序列为 A, B, C, D, E , 运用栈操作, 能否得到出栈序列 B, C, A, E, D 和 D, B, A, C, E ? 为什么?

03. 栈的初态和终态均为空, 以 I 和 O 分别表示入栈和出栈, 则出入栈的操作序列可表示为由 I 和 O 组成的序列, 可以操作的序列称为合法序列, 否则称为非法序列。

1) 下面所示的序列中哪些是合法的?

- A. IOIOIOIO B. IOOIOIOI C. IIIIOIOIO D. IIIOOIOIO

2) 通过对 1) 的分析, 写出一个算法, 判定所给的操作序列是否合法。若合法, 返回 true, 否则返回 false (假定被判定的操作序列已存入一维数组中)。

04. 设单链表的表头指针为 L , 结点结构由 data 和 next 两个域构成, 其中 data 域为字符型。设计算法判断该链表的全部 n 个字符是否中心对称。例如, xyx 、 $xyyx$ 都是中心对称的。

05. 设有两个栈 S_1 、 S_2 都采用顺序栈方式, 并共享一个存储区 $[0, \dots, \text{maxsize}-1]$, 为了尽量利用空间, 减少溢出的可能, 可采用栈顶相向、迎面增长的存储方式。试设计 S_1 、 S_2 有关入栈和出栈的操作算法。

3.1.5 答案与解析

一、单项选择题

01. B

栈和队列的逻辑结构都是相同的, 都属于线性结构, 只是它们对数据的运算不同。

02. C

首先栈是一种线性表, 所以选项 B、D 错。按存储结构的不同可分为顺序栈和链栈, 但不可以把栈局限在某种存储结构上, 所以选项 A 错。栈和队列都是限制存取点的线性结构。

03. B

基本操作是指该结构最核心、最基本的运算, 其他较复杂的操作可通过基本操作实现。删除栈底元素不属于栈的基本运算, 但它可以通过调用栈的基本运算求得。

04. C

数组下标范围为 $0 \sim n-1$ ，初始时 top 为 -1，第一个元素入栈后， top 为 0，即 top 指向栈顶元素。栈向高地址方向增长，所以入栈时应先将指针 top 加 1，然后存入元素 x ，选项 C 正确。

05. B

数组下标范围为 $1 \sim n$ ，初始时 top 为 1，表示 top 指向栈顶元素的下一个元素。栈向高地址方向增长，所以入栈时应先存入元素 x ，然后将指针 top 加 1，选项 B 正确。

06. A

数组下标范围为 $1 \sim n$ ，初始时 top 为 $n+1$ ，表示 top 指向栈顶元素。栈向低地址方向增长，所以入栈时应先将指针 top 减 1，然后存入元素 x ，A 正确。

07. A

每个元素需要 1 个存储单元，所以每入栈一次 top 加 1，出栈一次 top 减 1。指针 top 的值依次为 1001H, 1002H, 1001H, 1002H, 1001H, 1002H, 1001H, 1002H。

08. A

顺序栈采用数组存储，数组的大小是固定的，不能动态地分配大小。和顺序栈相比，链栈的最大优势在于它可以动态地分配存储空间。

09. C

对于双向循环链表，不管是表头指针还是表尾指针，都可以很方便地找到表头结点，方便在表头做插入或删除操作。而循环单链表通过尾指针可以很方便地找到表头结点，但通过头指针找尾结点需要遍历一次链表。对于选项 C，插入和删除结点后，找尾结点所需的时间为 $O(n)$ 。

10. C

链栈采用不带头结点的单链表表示时，入栈操作在首部插入一个结点 x ($x \rightarrow \text{next} = \text{top}$)，插入完后需将 top 指向该插入的结点 x 。请思考当链栈存在头结点时的情况。

11. D

这里假设栈顶指针指向的是栈顶元素，所以选择选项 D；而选项 A 中首先将 top 指针赋给了 x ，错误；选项 B 中没有修改 top 指针的值；选项 C 为 top 指针指向栈顶元素的上一个元素时的答案。

12. A

执行前 3 句后，栈 st 内的值为 a, b ，其中 b 为栈顶元素；执行第 4 句后，栈顶元素 b 出栈， x 的值为 b ；执行最后一句，读取栈顶元素的值， x 的值为 a 。

13. B

当 n 个不同元素入栈时，出栈序列的个数为

$$\frac{1}{n+1} C_{2n}^n = \frac{1}{n+1} \frac{(2n)!}{n! \times n!} = \frac{6 \times 5 \times 4}{4 \times 3 \times 2 \times 1} = 5$$

考题中给出的 n 值不会很大，可以根据栈的特点，若 X_i 已经出栈，则 X_i 前面的尚未出栈的元素一定逆置有序地出栈，因此，可以采用列举方法。如 a, b, c 依次入栈的出栈序列有 abc, acb, bac, bca, cba 。另外，在一些考题中可能会问符合某个特定条件的出栈序列有多少种，比如此题中间以 b 开头的出栈序列有几种，这种类型的题目一般都使用穷举法。

14. D

根据栈“先进后出”的特点，且在入栈操作的同时允许出栈操作，显然选项 D 中 c 最先出栈，则此时栈内必定为 a 和 b ，但因为 a 先于 b 入栈，所以要晚出栈。对于某个出栈的元素，在它之前入栈却晚出栈的元素必定是按逆序出栈的，其余答案均是可能出现的情况。

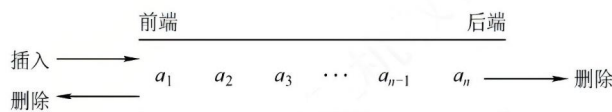


图 3.12 输入受限的双端队列

例 设有一个双端队列，输入序列为 1, 2, 3, 4，试分别求出以下条件的输出序列。

- (1) 能由输入受限的双端队列得到，但不能由输出受限的双端队列得到的输出序列。
- (2) 能由输出受限的双端队列得到，但不能由输入受限的双端队列得到的输出序列。
- (3) 既不能由输入受限的双端队列得到，又不能由输出受限的双端队列得到的输出序列。

解：先看输入受限的双端队列，见图 3.13。假设 end1 端输入 1, 2, 3, 4，则 end2 端的输出相当于普通队列的输出，即 1, 2, 3, 4；而 end1 端的输出相当于栈的输出， $n=4$ 时仅通过 end1 端有 14 种输出序列（由 Catalan 公式得出），仅通过 end1 端不能得到的输出序列有 $4! - 14 = 10$ 种：

1, 4, 2, 3 2, 4, 1, 3 3, 4, 1, 2 3, 1, 4, 2 3, 1, 2, 4
 4, 3, 1, 2 4, 1, 3, 2 4, 2, 3, 1 4, 2, 1, 3 4, 1, 2, 3

通过 end1 和 end2 端混合输出，可以输出这 10 种中的 8 种，参见下表。其中， S_L, X_L 分别代表 end1 端的入队和出队， X_R 代表 end2 端的出队。

输出序列	入队出队顺序	输出序列	入队出队顺序
1, 4, 2, 3	$S_L X_R S_L S_L X_L X_R X_R$	3, 1, 2, 4	$S_L S_L S_L X_L X_R X_R X_R$
2, 4, 1, 3	$S_L S_L X_L S_L S_L X_L X_R X_R$	4, 1, 2, 3	$S_L S_L S_L S_L X_R X_R X_R$
3, 4, 1, 2	$S_L S_L S_L X_L S_L X_L X_R X_R$	4, 1, 3, 2	$S_L S_L S_L S_L X_R X_L X_R$
3, 1, 4, 2	$S_L S_L S_L X_L X_R S_L X_L X_R$	4, 3, 1, 2	$S_L S_L S_L S_L X_L X_R X_R$

剩下的两种是不能通过输入受限的双端队列输出的，即 4, 2, 3, 1 和 4, 2, 1, 3。

再看输出受限的双端队列，见图 3.14。假设 end1 和 end2 端都能输入，仅 end2 端可以输出。若都从 end2 端输入，则就是一个栈。当输入序列为 1, 2, 3, 4 时，输出序列有 14 种。对于其余 10 种不能得到的输出序列，通过交替从 end1 和 end2 端输入，还可以输出其中 8 种。设 S_L 代表 end1 端的输入， S_R, X_R 分别代表 end2 端的输入和输出，则可能的输出序列见下表。



图 3.13 例题中输入受限的双端队列



图 3.14 例题中输出受限的双端队列

输出序列	入队出队顺序	输出序列	入队出队顺序
1, 4, 2, 3	$S_L X_R S_L S_L S_R X_R X_R X_R$	3, 1, 2, 4	$S_L S_L S_R X_R X_R S_L X_R X_R$
2, 4, 1, 3	$S_L S_R X_R S_L S_R X_R X_R X_R$	4, 1, 2, 3	$S_L S_L S_L S_R X_R X_R X_R X_R$
3, 4, 1, 2	$S_L S_L S_R X_R S_R X_R X_R X_R$	4, 2, 1, 3	$S_L S_R S_L S_R X_R X_R X_R X_R$
3, 1, 4, 2	$S_L S_L S_R X_R X_R S_R X_R X_R$	4, 3, 1, 2	$S_L S_L S_R S_R X_R X_R X_R X_R$

通过输出受限的双端队列不能得到的两种输出序列是 4, 1, 3, 2 和 4, 2, 3, 1。

综上所述：

- 1) 能由输入受限的双端队列得到，但不能由输出受限的双端队列得到的是 4, 1, 3, 2。
- 2) 能由输出受限的双端队列得到，但不能由输入受限的双端队列得到的是 4, 2, 1, 3。
- 3) 既不能由输入受限的双端队列得到，又不能由输出受限的双端队列得到的是 4, 2, 3, 1。

提示

实际双端队列的考题不会如此复杂,通常只需判断序列是否满足题设条件,代入验证即可。

3.2.5 本节试题精选

一、单项选择题

01. 栈和队列的主要区别在于 ()。
A. 它们的逻辑结构不一样
B. 它们的存储结构不一样
C. 所包含的元素不一样
D. 插入、删除操作的限定不一样
02. 队列的“先进先出”特性是指 ()。
I. 最后插入队列中的元素总是最后被删除
II. 当同时进行插入、删除操作时,总是插入操作优先
III. 每当有删除操作时,总要先做一次插入操作
IV. 每次从队列中删除的总是最早插入的元素
A. I
B. I 和 IV
C. II 和 III
D. IV
03. 允许对队列进行的操作有 ()。
A. 对队列中的元素排序
B. 取出最近入队的元素
C. 在队列元素之间插入元素
D. 删除队首元素
04. 一个队列的入队顺序是 1, 2, 3, 4, 则出队的输出顺序是 ()。
A. 4, 3, 2, 1
B. 1, 2, 3, 4
C. 1, 4, 3, 2
D. 3, 2, 4, 1
05. 循环队列存储在数组 $A[0..n]$ 中, 入队时的操作为 ()。
A. $rear=rear+1$
B. $rear=(rear+1) \bmod (n-1)$
C. $rear=(rear+1) \bmod n$
D. $rear=(rear+1) \bmod (n+1)$
06. 已知循环队列的存储空间为数组 $A[21]$, $front$ 指向队首元素的前一个位置, $rear$ 指向队尾元素, 假设当前 $front$ 和 $rear$ 的值分别为 8 和 3, 则该队列的长度为 ()。
A. 5
B. 6
C. 16
D. 17
07. 若用数组 $A[0..5]$ 实现循环队列, 且当前 $rear$ 和 $front$ 的值分别为 1 和 5, 当从队列中删除一个元素, 再加入两个元素后, $rear$ 和 $front$ 的值分别为 ()。
A. 3 和 4
B. 3 和 0
C. 5 和 0
D. 5 和 1
08. 假设用数组 $Q[MaxSize]$ 实现循环队列, 队首指针 $front$ 指向队首元素的前一位置, 队尾指针 $rear$ 指向队尾元素, 则判断该队列为空的条件是 ()。
A. $Q.rear==(Q.front+1)\%MaxSize$
B. $(Q.rear+1)\%MaxSize==Q.front+1$
C. $(Q.rear+1)\%MaxSize==Q.front$
D. $Q.rear==Q.front$
09. 假设循环队列 $Q[MaxSize]$ 的队首指针为 $front$, 队尾指针为 $rear$, 队列的最大容量为 $MaxSize$, 此外, 该队列再没有其他数据成员, 则判断该队列已满的条件是 ()。
A. $Q.front==Q.rear$
B. $Q.front+Q.rear>=MaxSize$
C. $Q.front==(Q.rear+1)\%MaxSize$
D. $Q.rear==(Q.front+1)\%MaxSize$
10. 假设用 $A[0..n]$ 实现循环队列, $front$ 、 $rear$ 分别指向队首元素的前一个位置和队尾元素。若用 $(rear+1)\%(n+1)==front$ 作为队满标志, 则 ()。

- A. 可用 $\text{front} == \text{rear}$ 作为队空标志
 - B. 队列中最多可有 $n+1$ 个元素
 - C. 可用 $\text{front} > \text{rear}$ 作为队空标志
 - D. 可用 $(\text{front} + 1) \% (n + 1) == \text{rear}$ 作为队空标志
11. 与顺序队列相比, 链式队列的 ()。
- A. 优点是队列的长度不受限制
 - B. 优点是入队和出队时间效率更高
 - C. 缺点是不能进行顺序访问
 - D. 缺点是不能根据队首指针和队尾指针计算队列的长度
12. 下列描述的几种链表中, 最适合用作队列的是 ()。
- A. 带队首指针和队尾指针的循环单链表
 - B. 带队首指针和队尾指针的非循环单链表
 - C. 只带队首指针的非循环单链表
 - D. 只带队首指针的循环单链表
13. 下列描述的几种链表中, 最不适合用作链式队列的是 ()。
- A. 只带队首指针的非循环双链表
 - B. 只带队首指针的循环双链表
 - C. 只带队尾指针的循环双链表
 - D. 只带队尾指针的循环单链表
14. 在用单链表实现队列时, 队头设在链表的 () 位置。
- A. 链头
 - B. 链尾
 - C. 链中
 - D. 以上都可以
15. 用链式存储方式的队列进行删除操作时需要 ()。
- A. 仅修改头指针
 - B. 仅修改尾指针
 - C. 头尾指针都要修改
 - D. 头尾指针可能都要修改
16. 在一个链式队列中, 假设队首指针为 front , 队尾指针为 rear , x 所指向的元素需要入队, 则需要执行的操作为 ()。
- A. $\text{front} = x, \text{front} = \text{front} \rightarrow \text{next}$
 - B. $x \rightarrow \text{next} = \text{front} \rightarrow \text{next}, \text{front} = x$
 - C. $\text{rear} \rightarrow \text{next} = x, \text{rear} = x$
 - D. $\text{rear} \rightarrow \text{next} = x, x \rightarrow \text{next} = \text{NULL}, \text{rear} = x$
17. 假设循环单链表表示的队列长度为 n , 队头固定在链表尾, 若只设头指针, 则入队操作的时间复杂度为 ()。
- A. $O(n)$
 - B. $O(1)$
 - C. $O(n^2)$
 - D. $O(n \log_2 n)$
18. 假设输入序列为 1, 2, 3, 4, 5, 利用两个队列进行出入队操作, 不可能输出的序列是 ()。
- A. 1, 2, 3, 4, 5
 - B. 5, 2, 3, 4, 1
 - C. 1, 3, 2, 4, 5
 - D. 4, 1, 5, 2, 3
19. 若以 1, 2, 3, 4 作为双端队列的输入序列, 则既不能由输入受限的双端队列得到, 又不能由输出受限的双端队列得到的输出序列是 ()。
- A. 1, 2, 3, 4
 - B. 4, 1, 3, 2
 - C. 4, 2, 3, 1
 - D. 4, 2, 1, 3
20. 【2010 统考真题】某队列允许在其两端进行入队操作, 但仅允许在一端进行出队操作。若元素 a, b, c, d, e 依次入此队列后再进行出队操作, 则不可能得到的出队序列是 ()。
- A. b, a, c, d, e
 - B. d, b, a, c, e
 - C. d, b, c, a, e
 - D. e, c, b, a, d
21. 【2011 统考真题】已知循环队列存储在一维数组 $A[0 \dots n-1]$ 中, 且队列非空时 front 和 rear 分别指向队首元素和队尾元素。若初始时队列为空, 且要求第一个进入队列的

元素存储在 $A[0]$ 处, 则初始时 $front$ 和 $rear$ 的值分别是 ()。

- A. 0, 0 B. 0, $n-1$ C. $n-1$, 0 D. $n-1$, $n-1$

22. 【2014 统考真题】循环队列放在一维数组 $A[0..M-1]$ 中, $end1$ 指向队首元素, $end2$ 指向队尾元素的后一个位置。假设队列两端均可进行入队和出队操作, 队列中最多能容纳 $M-1$ 个元素。初始时空。下列判断队空和队满的条件中, 正确的是 ()。

- A. 队空: $end1 == end2$; 队满: $end1 == (end2 + 1) \bmod M$
 B. 队空: $end1 == end2$; 队满: $end2 == (end1 + 1) \bmod (M - 1)$
 C. 队空: $end2 == (end1 + 1) \bmod M$; 队满: $end1 == (end2 + 1) \bmod M$
 D. 队空: $end1 == (end2 + 1) \bmod M$; 队满: $end2 == (end1 + 1) \bmod (M - 1)$

23. 【2018 统考真题】现有队列 Q 与栈 S , 初始时 Q 中的元素依次是 1, 2, 3, 4, 5, 6 (1 在队头), S 为空。若仅允许下列 3 种操作: ① 出队并输出出队元素; ② 出队并将出队元素入栈; ③ 出栈并输出出栈元素, 则不能得到的输出序列是 ()。

- A. 1, 2, 5, 6, 4, 3 B. 2, 3, 4, 5, 6, 1 C. 3, 4, 5, 6, 1, 2 D. 6, 5, 4, 3, 2, 1

24. 【2021 统考真题】初始为空的队列 Q 的一端仅能进行入队操作, 另外一端既能进行入队操作又能进行出队操作。若 Q 的入队序列是 1, 2, 3, 4, 5, 则不能得到的出队序列是 ()。

- A. 5, 4, 3, 1, 2 B. 5, 3, 1, 2, 4 C. 4, 2, 1, 3, 5 D. 4, 1, 3, 2, 5

二、综合应用题

01. 若希望循环队列中的元素都能得到利用, 则需设置一个标志域 tag , 并以 tag 的值为 0 或 1 来区分队首指针 $front$ 和队尾指针 $rear$ 相同时的队列状态是“空”还是“满”。试编写与此结构相应的入队和出队算法。

02. Q 是一个队列, S 是一个空栈, 实现将队列中的元素逆置的算法。

03. 利用两个栈 $S1$ 和 $S2$ 来模拟一个队列, 已知栈的 4 个运算定义如下:

$Push(S, x);$	//元素 x 入栈 S
$Pop(S, x);$	// S 出栈并将出栈的值赋给 x
$StackEmpty(S);$	//判断栈是否为空
$StackOverflow(S);$	//判断栈是否为满

如何利用栈的运算来实现该队列的 3 个运算 (形参由读者根据要求自己设计)?

$Enqueue;$	//将元素 x 入队
$Dequeue;$	//出队, 并将出队元素存储在 x 中
$QueueEmpty;$	//判断队列是否为空

04. 【2019 统考真题】请设计一个队列, 要求满足: ① 初始时队列为空; ② 入队时, 允许增加队列占用空间; ③ 出队后, 出队元素所占用的空间可重复使用, 即整个队列所占用的空间只增不减; ④ 入队操作和出队操作的时间复杂度始终保持为 $O(1)$ 。请回答:

- 1) 该队列是应选择链式存储结构, 还是应选择顺序存储结构?
- 2) 画出队列的初始状态, 并给出判断队空和队满的条件。
- 3) 画出第一个元素入队后的队列状态。
- 4) 给出入队操作和出队操作的基本过程。

3.2.6 答案与解析

一、单项选择题

01. D

栈和队列的逻辑结构都是线性结构, 都可以采用顺序存储或链式存储, 选项 A 和 B 错误,

选项 C 显然也错误。只有选项 D 才是栈和队列的本质区别,限定表中插入和删除操作位置的不同。

02. B

队列“先进先出”的特性表现在:先入队列的元素先出队列,后入队列的元素后出队列,入队对应的是插入操作,出队对应的是删除操作。说法 I 和 IV 均正确。

03. D

删除队首元素即出队,是队列的基本操作之一,所以选择选项 D。

04. B

队列的入队顺序和出队顺序是一致的,这是和栈不同的。

05. D

数组下标范围为 $0 \sim n$,因此数组容量为 $n+1$ 。循环队列中元素入队的操作是 $\text{rear}=(\text{rear}+1) \bmod \text{maxsize}$,题中 $\text{maxsize}=n+1$ 。因此入队操作应为 $\text{rear}=(\text{rear}+1) \bmod (n+1)$ 。

06. C

队列的长度为 $(\text{rear}-\text{front}+\text{maxsize})\% \text{maxsize}=(\text{rear}-\text{front}+21)\%21=16$ 。这种情况和 front 指向当前元素, rear 指向队尾元素的下一个元素的计算是相同的。

注意

数组 $A[n]$ 的下标范围为 $0 \sim n-1$ 。若写成 $A[0..n]$,则说明下标范围为 $0 \sim n$ 。

07. B

循环队列中,每删除一个元素,队首指针 $\text{front}=(\text{front}+1)\%6$,每插入一个元素,队尾指针 $\text{rear}=(\text{rear}+1)\%6$ 。上述操作后, $\text{front}=0$, $\text{rear}=3$ 。

08. D

当队列中只有一个元素时, front 指向该元素的前一个位置, rear 指向该元素,因此当队列为空时,队首指针等于队尾指针,这样第一个元素入队后,才能符合题目要求。

09. C

既然不能附加任何其他数据成员,只能采用牺牲一个存储单元的方法来区分是队空还是队满,约定以“队列头指针在队尾指针的下一位置作为队满的标志”,因此选择选项 C。选项 A 是判断队列是否空的条件,选项 B 和 D 都是干扰项。

注意

对于这类具体问题,举一些特例判断往往比直接思考问题能更快得到答案。

10. A

若用 $(\text{rear}+1)\%(n+1) == \text{front}$ 作为队满标志,则说明题目采用了牺牲一个存储单元的方法来区分队空和队满,因此可用 $\text{front} == \text{rear}$ 作为队空标志。

11. D

虽然链式队列采用动态分配方式,但其长度也受内存空间的限制,不能无限制增长。顺序队列和链式队列的入队和出队时间复杂度均为 $O(1)$ 。顺序队列和链式队列都可以进行顺序访问。对于顺序队列,可通过队首指针和队尾指针计算队列中的元素个数,而链式队列则不能。

12. B

因为队列需在双端进行操作,所以选项 C 和 D 的链表显然不太适合链队。对于选项 A,链表在完成入队和出队后还要修改为循环的,对于队列来讲这是多余的(画蛇添足)。对于选项 B,

- **边界条件（递归出口）**：确保递归在有限步内终止。

递归的精髓在于能否将原问题转化为属性相同但规模更小的问题。

考点追踪 ▶ 栈在函数调用中的作用和工作原理（2015、2017）

在递归调用过程中，系统为每一层调用的返回地址、局部变量及形参等信息分配存储空间，这些信息被保存在递归工作栈中。因此，递归深度过大时容易导致**栈溢出**。此外，递归效率低下的主要原因是**存在大量重复计算**。以 $n=5$ 为例，其递归调用过程如图 3.16 所示。

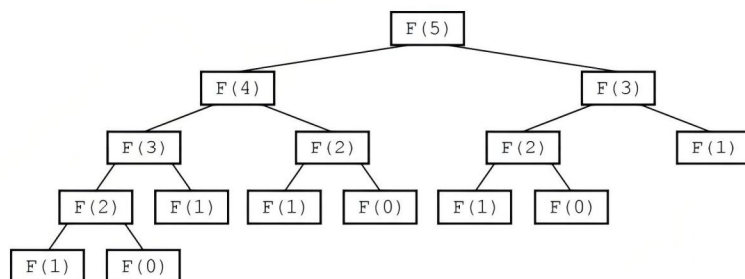


图 3.16 $F(5)$ 的递归执行过程

显然，在该过程中， $F(3)$ 被计算 2 次， $F(2)$ 被计算 3 次。 $F(1)$ 被调用 5 次， $F(0)$ 被调用 3 次。由此可见，递归虽然代码简洁、易于理解，但存在明显的性能缺陷。在第 5 章讨论树的遍历时，采用递归可使代码极为简洁，但初学者往往难以清晰理解其执行过程。若希望深入理解递归的底层实现机制，可参考《编译原理》教材中关于运行时栈与活动记录的相关内容。

任何递归算法均可转换为非递归算法，通常需借助显式栈来模拟递归调用过程。

3.3.4 队列在层次遍历中的应用

在信息处理中，有一类典型问题需要逐层或逐行处理。解决这类问题的常用策略是：在处理当前层的同时，将下一层的元素按序加入待处理队列。队列正适用于此类场景，因其先进先出的特性，能够自然地保存后续待处理元素的顺序。以二叉树的层次遍历为例（见图 3.17），可清晰体现队列的应用价值。表 3.3 展示了该遍历过程的具体执行步骤。

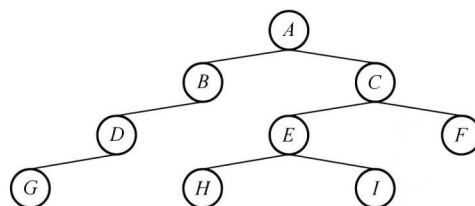


图 3.17 二叉树

表 3.3 层次遍历二叉树的过程

序 号	说 明	队 内	队 外
1	A 入	A	
2	A 出, BC 入	BC	A
3	B 出, D 入	CD	AB
4	C 出, EF 入	DEF	ABC
5	D 出, G 入	EFG	ABCD
6	E 出, HI 入	FGHI	ABCDE
7	F 出	GHI	ABCDEF
8	GHI 出		ABCDEFGHI

该过程可简要描述如下：

- ① 将根结点入队。

- ② 若队列为空（表示所有结点均已处理），则遍历结束；否则执行③。
③ 取出队首结点并访问；若其存在左孩子，则将左孩子入队；若其存在右孩子，则将右孩子入队；返回②继续执行。

3.3.5 队列在计算机系统中的应用

队列在计算机系统中的应用非常广泛，以下从两个主要方面进行阐述：第一个方面是解决主机与外部设备之间速度不匹配的问题，第二个方面是解决由多用户引起的资源竞争问题。

考点追踪 ▶ 缓冲区的逻辑结构（2009）

以主机和打印机之间的速度不匹配为例。主机输出数据的速度远快于打印机处理数据的速度。若直接将数据发送给打印机，会导致数据丢失或打印错误。为此，通常设置一个打印数据缓冲区来缓解这一问题。具体实现如下：主机将待打印的数据依次写入缓冲区，当缓冲区满时，主机暂停输出并转向其他任务；打印机则从缓冲区中按先进先出原则逐个取出数据进行打印；打印完成后，打印机向主机发出请求，主机再次向缓冲区写入新的打印数据。这种方法不仅保证了数据的正确性，还提高了主机的整体效率。因此，打印数据缓冲区实际上就是一个队列。

考点追踪 ▶ 多队列出队/入队操作的应用（2016）

在多用户环境下，CPU 资源的竞争是一个典型场景。在一个多终端系统中，用户通过各自的终端向操作系统提出对 CPU 的请求。为公平分配 CPU 时间，操作系统通常按照请求的时间顺序，将这些请求排成一个队列。具体步骤如下：每次将 CPU 分配给队首用户的程序运行；当该程序运行结束或用完规定的时间片后，操作系统使其出队；然后将 CPU 分配给新的队首用户。这种方式既保证了请求的公平处理，又提升了 CPU 利用率。此外，在某些复杂系统中，可能还会引入多队列机制，以便根据不同的优先级或调度策略动态调整资源分配。

3.3.6 本节试题精选

一、单项选择题

01. 栈的应用不包括（ ）。
A. 递归 B. 表达式求值 C. 括号匹配 D. 缓冲区
02. 表达式 $a*(b+c)-d$ 的后缀表达式是（ ）。
A. $abcd*+-$ B. $abc+*d-$ C. $abc*+d-$ D. $-+*abcd$
03. 下面（ ）用到了队列。
A. 括号匹配 B. 表达式求值 C. 递归 D. FIFO 页面替换算法
04. 利用栈求表达式的值时，设立运算数栈 OPEN。假设 OPEN 只有两个存储单元，则在下列表达式中，不会发生溢出的是（ ）。
A. $A-B*(C-D)$ B. $(A-B)*C-D$ C. $(A-B*C)-D$ D. $(A-B)*(C-D)$
05. 执行下列语句段后， i 的值为（ ）。

```
int f(int x){  
    return ((x>0)? x*f(x-1):2);  
}  
int i;  
i=f(f(1));
```

A. 2

B. 4

C. 8

D. 无限递归

06. 设有如下递归函数, 则计算 $F(8)$ 需要调用该递归函数的次数为 ()。

```
int F(int n){
    if(n<=3) return 1;
    else return F(n-2)+F(n-4)+1;
}
```

- A. 7 B. 8 C. 9 D. 10

07. 设有如下递归函数, 在 $\text{func}(\text{func}(5))$ 的执行过程中, 第4个被执行的 func 函数是 ()。

```
int func(int x){
    if(x<=3) return 2;
    else return func(x-2)+func(x-4);
}
```

- A. $\text{func}(2)$ B. $\text{func}(3)$ C. $\text{func}(4)$ D. $\text{func}(5)$

08. 对于一个问题的递归算法求解和其相对应的非递归算法求解, ()。

- A. 递归算法通常效率高一些 B. 非递归算法通常效率高一些
C. 两者相同 D. 无法比较

09. 执行函数时, 其局部变量一般采用 () 进行存储。

- A. 树形结构 B. 静态链表 C. 栈结构 D. 队列结构

10. 执行 () 操作时, 需要使用队列作为辅助存储空间。

- A. 查找散列 (哈希) 表 B. 广度优先搜索图
C. 前序 (根) 遍历二叉树 D. 深度优先搜索图

11. 下列说法中, 正确的是 ()。

- A. 消除递归不一定需要使用栈
B. 对同一输入序列进行两组不同的合法入栈和出栈组合操作, 所得的输出序列也一定相同
C. 通常使用队列来处理函数或过程调用
D. 队列和栈都是运算受限的线性表, 只允许在表的两端进行运算

12. 【2009 统考真题】为解决计算机主机与打印机之间速度不匹配的问题, 通常设置一个打印数据缓冲区, 主机将要输出的数据依次写入该缓冲区, 而打印机则依次从该缓冲区中取出数据。该缓冲区的逻辑结构应该是 ()。

- A. 栈 B. 队列 C. 树 D. 图

13. 【2012 统考真题】已知操作符包括 “+” “-” “*” “/” “(” 和 “)”。将中缀表达式 $a+b-a*((c+d)/e-f)+g$ 转换为等价的后缀表达式 $ab+acd+e/f-* -g+$ 时, 用栈来存放暂时还不能确定运算次序的操作符。栈初始时空, 转换过程中同时保存在栈中的操作符的最大个数是 ()。

- A. 5 B. 7 C. 8 D. 11

14. 【2014 统考真题】假设栈初始为空, 将中缀表达式 $a/b+(c*d-e*f)/g$ 转换为等价的后缀表达式过程中, 当扫描到 f 时, 栈中的元素依次是 ()。

- A. $+(*-$ B. $+(-*$ C. $/+(*-*$ D. $/+ -*$

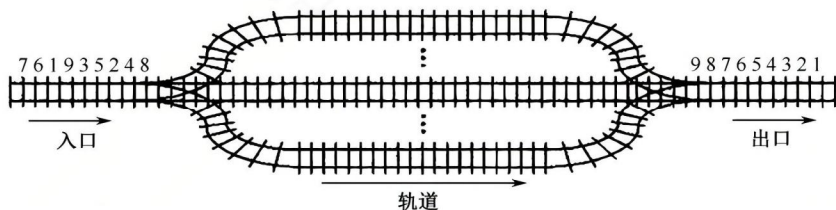
15. 【2015 统考真题】已知程序如下:

```
int S(int n)
{
    return (n<=0)?0:S(n-1)+n;
}
void main()
{
    cout<<S(1);
}
```

程序运行时使用栈来保存调用过程的信息, 自栈底到栈顶保存的信息依次对应的是 ()。

- A. $\text{main}() \rightarrow S(1) \rightarrow S(0)$ B. $S(0) \rightarrow S(1) \rightarrow \text{main}()$
 C. $\text{main}() \rightarrow S(0) \rightarrow S(1)$ D. $S(1) \rightarrow S(0) \rightarrow \text{main}()$

16. 【2016 统考真题】设有如下图所示的火车车轨，入口和出口之间有 n 条轨道，列车的行进方向均为从左至右，列车可驶入任意一条轨道。现有编号为 1~9 的 9 列列车，驶入的次序依次是 8, 4, 2, 5, 3, 9, 1, 6, 7。若期望驶出的次序依次为 1~9，则 n 至少是 ()。



- A. 2 B. 3 C. 4 D. 5
17. 【2017 统考真题】下列关于栈的叙述中，错误的是 ()。
- I. 采用非递归方式重写递归程序时必须使用栈
 - II. 函数调用时，系统要用栈保存必要的信息
 - III. 只要确定了入栈次序，即可确定出栈次序
 - IV. 栈是一种受限的线性表，允许在其两端进行操作
- A. 仅 I B. 仅 I、II、III C. 仅 I、III、IV D. 仅 II、III、IV
18. 【2018 统考真题】若栈 S1 中保存整数，栈 S2 中保存运算符，函数 F() 依次执行下述各步操作：
- 1) 从 S1 中依次弹出两个操作数 a 和 b。
 - 2) 从 S2 中弹出一个运算符 op。
 - 3) 执行相应的运算 $b \text{ op } a$ 。
 - 4) 将运算结果压入 S1 中。
- 假定 S1 中的操作数依次是 5, 8, 3, 2 (2 在栈顶)，S2 中的运算符依次是 *, -, + (+ 在栈顶)。调用 3 次 F() 后，S1 栈顶保存的值是 ()。
- A. -15 B. 15 C. -20 D. 20
19. 【2024 统考真题】与表达式 $x+y*(z-u)/v$ 等价的后缀表达式是 ()。
- A. $xyzuv-*v/+$ B. $xyzuv-v/*+$ C. $+x/y-zuv$ D. $+x*y/-zuv$
20. 【2025 统考真题】已知算法 A 用于检查字符串中各类括号是否匹配，A 执行过程中使用初始为空的栈保存遇到的括号。若栈的容量是 3，则下列选项中，A 不能处理的是 ()。
- A. $(a+[b+(c+d)/e]+f)+g-h$ B. $[a*((b+c)/(d-e)+f/g)-h]$
 C. $[a*(b-(c-d)*e/(f+g))-h]$ D. $[a-(b+[c*(d+e)-f]+g+h)]$

二、综合应用题

01. 假设一个算术表达式中包含圆括号、方括号和花括号 3 种类型的括号，编写一个算法来判别表达式中的括号是否配对，以字符 “\0” 作为算术表达式的结束符。

3.3.7 答案与解析

一、单项选择题

01. D

缓冲区是用队列实现的，选项 A、B、C 都是栈的典型应用。

02. B

后缀表达式中，每个运算符均直接位于其两个操作数的后面，按照这样的方式逐步根据计算的优先级将每个计算式进行变换，即可得到后缀表达式。

【另解】将两个直接操作数用括号括起来，再将操作符提到括号后，最后去掉括号。例如，对于 $(\textcircled{1})(\textcircled{2}a * (\textcircled{3}b + c)) - d$ ，提出操作符并去掉括号后，可得后缀表达式为 $abc + * d -$ 。

学完第5章后，可将表达式画成二叉树的形式，再用后序遍历即可求得后缀表达式。

03. D

FIFO 页面替换算法用到了队列。其余的都只用到了栈。

04. B

利用栈求表达式的值时，可以分别设立运算符栈和运算数栈，其原理不变。选项 B 中 A 入栈，B 入栈，计算得 R1，C 入栈，计算得 R2，D 入栈，计算得 R3，由此得栈深为 2。选项 A、C、D 依次计算得栈深为 4、3、3。因此选择选项 B。

技巧

根据运算符优先级，统计已依次入栈但还未参与计算的运算符数。以选项 C 为例，当 “(” “A” “-” 入栈时，“(” 和 “-” 还未参与运算，此时运算符栈大小为 2，“B” 和 “*” 入栈时运算符大小为 3，“C” 入栈时 “B*C” 运算，运算符栈大小为 2，以此类推。

05. B

栈与递归有着紧密的联系。递归模型包括递归出口和递归体两个方面。递归出口是递归算法的出口，即终止递归的条件。递归体是一个递推的关系式。根据题意有

$$f(0) = 2;$$

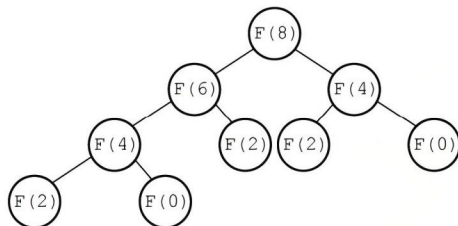
$$f(1) = 1 * f(0) = 2;$$

$$f(f(1)) = f(2) = 2 * f(1) = 4。$$

06. C

计算 $F(8)$ 的递归调用树如右图所示：

由图可知，递归函数 $F()$ 调用的次数为 9。

**07. C**

首先，执行内层参数 $\text{func}(5) = \text{func}(3) + \text{func}(1) = 4$ ，共执行 3 次 func 函数。然后，执行 $\text{func}(\text{func}(5)) = \text{func}(4) = \text{func}(2) + \text{func}(0) = 4$ ，因此第 4 个被执行的 func 函数是 $\text{func}(4)$ 。也可以采用画出递归调用树的方式，即某个函数的执行次序等于其在递归调用树的先序遍历中的次序。

08. B

通常情况下，递归算法在计算机实际执行的过程中包含很多的重复计算，所以效率会低。

09. C

调用函数时，系统会为调用者构造一个由参数表和返回地址组成的活动记录，并将记录压入系统提供的栈中，若被调用函数有局部变量，也要压入栈中。

10. B

本题涉及第5章和第6章的内容，图的广度优先搜索类似于树的层序遍历，都要借助于队列。

11. A

使用栈可以模拟递归的过程，以此来消除递归，但对于单向递归和尾递归而言，可以用迭代的方式来消除递归，选项 A 正确。不同的入栈和出栈组合操作，会产生许多不同的输出序列，B

对于元素 a_{ij} ($i \leq j$)，其在数组 B 中前方的元素个数为

第 1 行: n 个元素

第 2 行: $n-1$ 个元素

.....

第 $i-1$ 行: $n-i+2$ 个元素

第 i 行: $j-i$ 个元素

故元素 a_{ij} 在数组 B 中的下标 $k = n + (n-1) + \dots + (n-i+2) + (j-i+1) - 1 = (i-1)(2n-i+2)/2 + (j-i)$ 。元素下标之间的对应关系如下:

$$k = \begin{cases} \frac{(i-1)(2n-i+2)}{2} + (j-i), & i \leq j \text{ (上三角区和主对角线元素)} \\ \frac{n(n+1)}{2}, & i > j \text{ (下三角区元素)} \end{cases}$$

上三角矩阵的压缩存储形式如图 3.23 所示。

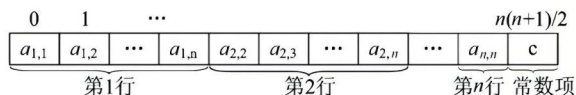


图 3.23 上三角矩阵的压缩存储

以上推导均假设数组下标从 0 开始。若下标指定从 1 开始，则要相应地调整映射关系。

3. 三对角矩阵

若 n 阶矩阵 A 中的任意元素 a_{ij} ，当 $|i-j| > 1$ 时均有 $a_{ij} = 0$ ($1 \leq i, j \leq n$)，则称为三对角矩阵，如图 3.24 所示。非零元素仅集中在以主对角线为中心的 3 条对角线的区域。

$$\begin{bmatrix} a_{1,1} & a_{1,2} & & & & & \\ a_{2,1} & a_{2,2} & a_{2,3} & & & & 0 \\ & a_{3,2} & a_{3,3} & a_{3,4} & & & \\ & & \ddots & \ddots & \ddots & & \\ & & & \ddots & \ddots & \ddots & \\ & 0 & & a_{n-1,n-2} & a_{n-1,n-1} & a_{n-1,n} & \\ & & & & a_{n,n-1} & a_{n,n} & \end{bmatrix}$$

图 3.24 三对角矩阵 A

可将三对角上的元素按行优先顺序存入一维数组 B，且 $a_{1,1}$ 存于 B[0]，如图 3.25 所示。

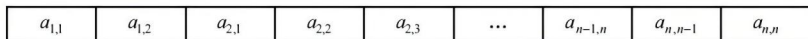


图 3.25 三对角矩阵的压缩存储

考点追踪 ▶▶ 三对角矩阵压缩存储的下标对应关系 (2016)

三对角上的元素 a_{ij} ($1 \leq i, j \leq n, |i-j| \leq 1$) 在一维数组 B 中的下标 $k = 2i + j - 3$ 。

反之，已知元素存于 B[k] 中时， $i = \lfloor (k+1)/3 \rfloor + 1$ ， $j = k - 2i + 3$ 。例如，当 $k=0$ 时， $i = \lfloor (0+1)/3 \rfloor + 1 = 1$ ， $j = 0 - 2 \times 1 + 3 = 1$ ，存放的是 $a_{1,1}$ ；当 $k=2$ 时， $i = \lfloor (2+1)/3 \rfloor + 1 = 2$ ， $j = 2 - 2 \times 2 + 3 = 1$ ，存放的是 $a_{2,1}$ ；当 $k=4$ 时， $i = \lfloor (4+1)/3 \rfloor + 1 = 2$ ， $j = 4 - 2 \times 2 + 3 = 3$ ，存放的是 $a_{2,3}$ 。

3.4.4 稀疏矩阵

若矩阵中非零元素的个数 t 相对于总元素个数 s 来说非常少，即 $t \ll s$ ，则称该矩阵为稀疏矩阵。

阵。例如, 一个 100×100 的矩阵中只有不到 100 个非零元素。

考点追踪 ▶ 存储稀疏矩阵需要保存的信息 (2023)

为避免空间浪费, 稀疏矩阵通常仅存储非零元素。然而, 由于非零元素的分布通常是无规律的, 仅存储其值是不够的, 还需记录它们所在的行和列。为此, 将每个非零元素及其对应的行列位置组合成一个三元组 (行标 i , 列标 j , 值 a_{ij}), 如图 3.26 所示。这些三元组可以按某种顺序排列成线性表进行存储。稀疏矩阵压缩存储后便失去了随机存取特性。

$$M = \begin{bmatrix} 4 & 0 & 0 & 0 \\ 0 & 0 & 6 & 0 \\ 0 & 9 & 0 & 0 \\ 0 & 23 & 0 & 0 \end{bmatrix}$$

对应的三元组

i	j	a_{ij}
0	0	4
1	2	6
2	1	9
3	1	23

图 3.26 稀疏矩阵及其对应的三元组

考点追踪 ▶ 稀疏矩阵压缩存储的方式 (2017)

稀疏矩阵的三元组表可通过多种方式存储, 常见的有: **数组存储**将所有三元组按某种顺序存储在一维数组中, 这种方式简单直接, 但插入和删除操作的效率较低; **十字链表存储** (见 6.2 节) 通过链表的方式组织三元组, 适用于频繁插入和删除操作的场景。无论采用哪种方式, 都需要额外保存稀疏矩阵的行数、列数和非零元素的个数, 以便支持后续的各种操作。

3.4.5 本节试题精选

单项选择题

01. 对特殊矩阵采用压缩存储的主要目的是 ()。

A. 表达变得简单
B. 对矩阵元素的存取变得简单

C. 去掉矩阵中的多余元素
D. 减少不必要的存储空间
02. 对 n 阶对称矩阵压缩存储时, 需要表长为 () 的顺序表。

A. $n/2$
B. $n \times n/2$
C. $n(n+1)/2$
D. $n(n-1)/2$
03. 有一个 $n \times n$ 的对称矩阵 A , 将其下三角部分按行存放在一维数组 B 中, 而 $A[0][0]$ 存放于 $B[0]$ 中, 则元素 $A[i][i]$ 存放于 B 中的 () 处。

A. $(i+3)i/2$
B. $(i+1)i/2$
C. $(2n-i+1)i/2$
D. $(2n-i-1)i/2$
04. 在二维数组 A 中, 假设每个数组元素的长度为 3 个存储单元, 行下标 i 为 $0 \sim 8$, 列下标 j 为 $0 \sim 9$, 从首地址 SA 开始连续存放。在这种情况下, 元素 $A[8][5]$ 的起始地址为 ()。

A. $SA + 141$
B. $SA + 144$
C. $SA + 222$
D. $SA + 255$
05. 二维数组 A 按行优先存储, 其中每个元素占 1 个存储单元。若 $A[1][1]$ 的存储地址为 420, $A[3][3]$ 的存储地址为 446, 则 $A[5][5]$ 的存储地址为 ()。

A. 472
B. 471
C. 458
D. 457
06. 将三对角矩阵即数组 $A[1 \dots 100][1 \dots 100]$ 按行优先存入一维数组 $B[1 \dots 298]$ 中, 数组 A 中元素 $A[66][65]$ 在数组 B 中的位置 k 为 ()。

A. 198
B. 195
C. 197
D. 196
07. 若将 n 阶上三角矩阵 A 按列优先级压缩存放在一维数组 $B[1 \dots n(n+1)/2 + 1]$ 中, 则存放到 $B[k]$ 中的非零元素 a_{ij} ($1 \leq i, j \leq n$) 的下标 i, j 与 k 的对应关系是 ()。

A. $i(i+1)/2 + j$
B. $i(i-1)/2 + j - 1$
C. $j(j-1)/2 + i$
D. $j(j-1)/2 + i - 1$

08. 若将 n 阶下三角矩阵 A 按列优先顺序压缩存放在一维数组 $B[1 \dots n(n+1)/2 + 1]$ 中, 则存放到 $B[k]$ 中的非零元素 a_{ij} ($1 \leq i, j \leq n$) 的下标 i, j 与 k 的对应关系是 ()。
- A. $(j-1)(2n-j+1)/2 + i - j$ B. $(j-1)(2n-j+2)/2 + i - j + 1$
 C. $(j-1)(2n-j+2)/2 + i - j$ D. $(j-1)(2n-j+1)/2 + i - j - 1$
09. 稀疏矩阵采用压缩存储后的缺点主要是 ()。
- A. 无法判断矩阵的行列数 B. 丧失随机存取的特性
 C. 无法由行、列值查找某个矩阵元素 D. 使矩阵元素之间的逻辑关系更复杂
10. 下列关于矩阵的说法中, 正确的是 ()。
- I. 在 n ($n > 3$) 阶三对角矩阵中, 每行都有 3 个非零元素
 II. 稀疏矩阵的特点是矩阵中的元素较少
- A. 仅 I B. 仅 II C. I 和 II D. 无正确项
11. 【2016 统考真题】有一个 100 阶的三对角矩阵 M , 其元素 m_{ij} ($1 \leq i, j \leq 100$) 按行优先依次压缩存入下标从 0 开始的一维数组 N 中。元素 $m_{30,30}$ 在 N 中的下标是 ()。
- A. 86 B. 87 C. 88 D. 89
12. 【2017 统考真题】适用于压缩存储稀疏矩阵的两种存储结构是 ()。
- A. 三元组表和十字链表 B. 三元组表和邻接矩阵
 C. 十字链表和二叉链表 D. 邻接矩阵和十字链表
13. 【2018 统考真题】设有一个 12×12 阶对称矩阵 M , 将其上三角部分的元素 m_{ij} ($1 \leq i \leq j \leq 12$) 按行优先存入 C 语言的一维数组 N 中, 元素 $m_{6,6}$ 在 N 中的下标是 ()。
- A. 50 B. 51 C. 55 D. 66
14. 【2020 统考真题】将一个 10×10 阶对称矩阵 M 的上三角部分的元素 m_{ij} ($1 \leq i \leq j \leq 10$) 按列优先存入 C 语言的一维数组 N 中, 元素 $m_{7,2}$ 在 N 中的下标是 ()。
- A. 15 B. 16 C. 22 D. 23
15. 【2021 统考真题】二维数组 A 按行优先方式存储, 每个元素占用 1 个存储单元。若元素 $A[0][0]$ 的存储地址是 100, $A[3][3]$ 的存储地址是 220, 则元素 $A[5][5]$ 的存储地址是 ()。
- A. 295 B. 300 C. 301 D. 306
16. 【2023 统考真题】若采用三元组表存储结构存储稀疏矩阵 M , 则除三元组表外, 下列数据中还需要保存的是 ()。
- I. M 的行数 II. M 中包含非零元素的行数
 III. M 的列数 IV. M 中包含非零元素的列数
- A. 仅 I、III B. 仅 I、IV C. 仅 II、IV D. I、II、III、IV

3.4.6 答案与解析

单项选择题

01. D

特殊矩阵中含有很多相同元素或零元素, 所以可采用压缩存储, 以节省存储空间。

02. C

只需存储其上三角或下三角部分 (含对角线), 元素个数为 $n + (n-1) + \dots + 1 = n(n+1)/2$ 。

03. A

此题要注意 3 个细节: 矩阵的最小下标为 0; 数组下标也是从 0 开始的; 矩阵按行优先存在

数组中。注意到此三点, 答案不难得到为选项 A。此外, 本类题建议采用特殊值代入法求解, 例如, $A[1][1]$ 对应的下标应为 2, 代入后只有选项 A 满足条件。

技巧

对于特殊三角矩阵压缩存储的题, 心中应有“平移”搬动的思想, 并结合草图, 这样会比较形象, 在计算时再注意矩阵和数组的起始下标, 就不容易出错。

04. D

二维数组计算地址 (按行优先顺序) 的公式为

$$\text{LOC}(i, j) = \text{LOC}(0, 0) + (im + j)L$$

其中, $\text{LOC}(0, 0) = \text{SA}$, 是数组存放的首地址; $L = 3$ 是每个数组元素的长度; $m = 9 - 0 + 1 = 10$ 是数组的列数。因此有 $\text{LOC}(8, 5) = \text{SA} + (8 \times 10 + 5) \times 3 = \text{SA} + 255$, 所以选择选项 D。

05. A

本题未直接给出数组 A 的行数和列数, 因此需要根据题目中的信息来推理。因为该二维数组按行优先存储, 且 $A[3][3]$ 的存储地址为 446, 所以 $A[3][1]$ 的存储地址为 444, 又 $A[1][1]$ 的存储地址为 420, 显然 $A[1][1]$ 和 $A[3][1]$ 正好相差 2 行, 所以该矩阵的列数为 12。而 $A[5][3]$ 和 $A[3][3]$ 正好相差 2 行, $A[5][5]$ 和 $A[5][3]$ 又相差 2 个元素, 所以 $A[5][5]$ 的存储地址是 $446 + 24 + 2 = 472$ 。

06. B

对于三对角矩阵, 将 $A[1 \dots n][1 \dots n]$ 压缩至 $B[1 \dots 3n - 2]$ 时, $a_{i,j}$ 与 b_k 的对应关系为 $k = 2i + j - 2$ 。则 A 中的元素 $A[66][65]$ 在数组 B 中的位置 k 为 $2 \times 66 + 65 - 2 = 195$ 。

07. C

按列优先存储, 所以元素 $a_{i,j}$ 前面有 $j - 1$ 列, 共有 $1 + 2 + 3 + \dots + j - 1 = j(j - 1)/2$ 个元素, 元素 $a_{i,j}$ 在第 j 列上是第 i 个元素, 数组 B 的下标是从 1 开始, 因此 $k = j(j - 1)/2 + i$ 。

08. B

按列优先存储, 故元素 $a_{i,j}$ 前有 $j - 1$ 列, 共有 $n + (n - 1) + \dots + (n - j + 2) = (j - 1)(2n - j + 2)/2$ 个元素, 元素 $a_{i,j}$ 是第 j 列上第 $i - j + 1$ 个元素, 数组 B 的下标从 1 开始, $k = (j - 1)(2n - j + 2)/2 + i - j + 1$ 。

09. B

稀疏矩阵通常采用三元组来压缩存储, 存储矩阵元素的行列下标和相应的值, 因此不能根据矩阵元素的行列下标快速定位矩阵元素, 失去了随机存取的特性。

10. D

在三对角矩阵中, 第 1 行和最后 1 行只有 2 个非零元, 其余各行均有 3 个非零元。稀疏矩阵的特点是矩阵中非零元的个数较少。

11. B

三对角矩阵如下所示。

$$\begin{bmatrix} a_{1,1} & a_{1,2} & & & & \\ a_{2,1} & a_{2,2} & a_{2,3} & & & 0 \\ & a_{3,2} & a_{3,3} & a_{3,4} & & \\ & & \ddots & \ddots & \ddots & \\ & 0 & & a_{n-1,n-2} & a_{n-1,n-1} & a_{n-1,n} \\ & & & & a_{n,n-1} & a_{n,n} \end{bmatrix}$$

采用压缩存储, 将 3 条对角线上的元素按行优先方式存放在一维数组 B 中, 且 $a_{1,1}$ 存放于 B[0] 中, 其存储形式如下所示: